

Aula Detalhada - Subtrator, Comparador, Shifter E ULA

Tema do dia: subtração em hardware, subtrator, borrow, subtração por complemento de 2, comparador de magnitude, shifters lógicos e aritméticos e ULA como bloco combinacional controlado

Aula na sequência: 18 **Objetivo:** entender como circuitos digitais fazem subtração, comparação, deslocamento de bits e seleção de operações em uma ULA, conectando esses blocos ao somador estudado na Aula 17.

1. Onde Esta Aula Entra No Estudo?

Na Aula 17, você estudou somadores:

```
half-adder  
full-adder  
ripple-carry adder  
carry-in  
carry-out  
overflow
```

Agora vamos usar essas ideias para construir blocos maiores.

O caminho desta aula é:

```
subtração de bits  
-> borrow  
-> subtrator  
-> subtração usando complemento de 2  
-> comparador  
-> shifter  
-> ULA
```

A ideia mais importante é esta:

```
um computador não cria um circuito completamente novo para cada operação  
ele reaproveita blocos
```

Por exemplo:

```
subtração pode ser feita com somador  
comparação pode usar subtração  
ULA usa somador, portas lógicas, comparador e shifter
```

2. Subtração Binária No Papel

Antes do circuito, lembre a subtração de 1 bit.

A	B	A - B	Diferença	Borrow
0	0	0	0	0
0	1	-1	1	1
1	0	1	1	0
1	1	0	0	0

O caso especial é:

0 - 1

Não dá para tirar 1 de 0 sem pedir emprestado da próxima coluna.

Então:

diferença = 1
borrow = 1

Em português, **borrow** é o "empréstimo".

Em circuito, ele é a saída que avisa:

precisei pegar 1 emprestado da próxima coluna

3. Half-Subtractor

3.1 O que é

O half-subtractor faz a subtração de 1 bit sem borrow de entrada.

Entradas:

A
B

Saídas:

D -> diferença
Bout -> borrow de saída

Ele calcula:

$A - B$

3.2 Tabela-verdade

A	B	D	Bout
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Observe:

D vale 1 quando A e B são diferentes
Bout vale 1 apenas quando $A=0$ e $B=1$

Logo:

$D = A \text{ XOR } B$
 $\text{Bout} = A' \cdot B$

Comparação com half-adder:

Bloco	Saída principal	Saída auxiliar
Half-adder	$S = A \text{ XOR } B$	$\text{Cout} = A \cdot B$
Half-subtractor	$D = A \text{ XOR } B$	$\text{Bout} = A' \cdot B$

A diferença e a soma usam XOR.

O que muda é o sinal auxiliar:

soma -> carry
subtração -> borrow

4. Full-Subtractor

4.1 Por que existe

Em uma subtração de vários bits, cada coluna pode receber borrow da coluna anterior.

Então não basta calcular:

$$A - B$$

É preciso calcular:

$$A - B - \text{Bin}$$

Bin é o borrow de entrada.

4.2 Entradas e saídas

Entradas:

A
B
Bin

Saídas:

D
Bout

4.3 Tabela-verdade

A	B	Bin	A - B - Bin	D	Bout
0	0	0	0	0	0
0	0	1	-1	1	1
0	1	0	-1	1	1
0	1	1	-2	0	1
1	0	0	1	1	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	-1	1	1

A diferença é:

$$D = A \text{ XOR } B \text{ XOR } \text{Bin}$$

O borrow de saída vale 1 quando a subtração daquela coluna precisou pedir emprestado.

Uma forma da expressão é:

$$\text{Bout} = A' \cdot B + A' \cdot \text{Bin} + B \cdot \text{Bin}$$

Você não precisa decorar essa expressão se a prova estiver mais focada em arquitetura de blocos.

Mas precisa entender a ideia:

$$\text{Bout} = 1 \text{ quando } B + \text{Bin} \text{ é maior do que } A \text{ naquela coluna}$$

5. Subtração Usando Somador E Complemento De 2

5.1 A ideia mais importante da aula

Na prática, é muito comum implementar subtração usando um somador.

Regra:

$$A - B = A + \text{complemento_de_2}(B)$$

E complemento de 2 é:

inverte B
soma 1

Então:

$$A - B = A + B' + 1$$

Isso é excelente para hardware, porque reaproveita o mesmo somador da Aula 17.

5.2 Como o circuito faz isso

Para transformar um somador em somador/subtrator, usamos um sinal de controle.

Chame esse sinal de:

SUB

Quando:

SUB = 0 -> fazer A + B

SUB = 1 -> fazer A - B

Para cada bit de **B**, o circuito faz:

B_entrada = B XOR SUB

E o carry-in inicial é:

C0 = SUB

Veja os dois casos.

5.3 Caso SUB = 0

B_entrada = B XOR 0

B_entrada = B

C0 = 0

Logo:

resultado = A + B + 0

resultado = A + B

5.4 Caso SUB = 1

B_entrada = B XOR 1

B_entrada = B'

C0 = 1

Logo:

resultado = A + B' + 1

resultado = A - B

Esse é um circuito muito importante.

Diagrama mental:

```

      SUB
      |
      v
B0 ----> XOR ----> B0_entrada ----+
B1 ----> XOR ----> B1_entrada ----+
B2 ----> XOR ----> B2_entrada ----+--> somador ripple-carry --> resultado
B3 ----> XOR ----> B3_entrada ----+

SUB -----> C0 do somador

```

Resumo:

SUB controla duas coisas:

1. inverte ou não inverte B
2. entra como +1 no carry-in inicial

6. Exemplo De Subtração Por Complemento De 2

Calcule em 4 bits:

0110 - 0011

Em decimal:

6 - 3 = 3

Agora pelo circuito:

A = 0110
B = 0011

Inverta B:

B' = 1100

Some 1:

B' + 1 = 1101

Agora some com A:

```
0110
+ 1101
-----
1 0011
```

Resultado armazenado em 4 bits:

0011

Carry-out final:

1

Para subtração sem sinal usando complemento de 2:

```
carry-out final = 1 -> não houve borrow
carry-out final = 0 -> houve borrow
```

Aqui:

```
6 - 3 = 3
não houve borrow
```

7. Exemplo Com Borrow Em Subtração Sem Sinal

Calcule em 4 bits:

0011 - 0101

Sem sinal:

3 - 5

Como 3 é menor que 5, esperamos borrow.

Pelo complemento de 2:

```
B = 0101
B' = 1010
B' + 1 = 1011
```

Some:

```
  0011
+ 1011
-----
  1110
```

Carry-out final:

0

Em subtração sem sinal:

carry-out final 0 indica borrow

Se você interpretar **1110** em complemento de 2:

$1110 = -2$

E isso bate com:

$3 - 5 = -2$

Mas cuidado:

sem sinal, o resultado negativo não cabe
com sinal, 1110 representa -2

8. Overflow Em Subtração Com Sinal

Para complemento de 2, a regra de overflow na subtração é:

A - B tem overflow quando A e B têm sinais diferentes
e o resultado fica com sinal diferente de A

Exemplo em 4 bits:

$0111 - 1111$

Interpretação:

0111 = +7
1111 = -1

Então:

$7 - (-1) = 8$

Mas em 4 bits com sinal o intervalo é:

-8 até +7

+8 não cabe.

Pelo circuito:

$A - B = A + B' + 1$

$B = 1111$

$B' = 0000$

$B' + 1 = 0001$

0111

+ 0001

1000

Resultado:

$1000 = -8$

Dois sinais diferentes na subtração e resultado com sinal diferente de A .

Então:

houve overflow com sinal

9. Comparador De Magnitude

9.1 O que é

Um comparador recebe dois números e informa a relação entre eles.

Para dois números A e B , ele pode gerar:

A > B
A = B
A < B

Em um comparador bem comportado, apenas uma dessas saídas deve valer 1 por vez.

Exemplo:

A = 10
B = 01

Sem sinal:

A = 2
B = 1

Logo:

A > B = 1
A = B = 0
A < B = 0

9.2 Comparação começa pelo bit mais significativo

Para comparar números binários sem sinal, olhe do bit mais significativo para o menos significativo.

Exemplo:

A = 1010
B = 0111

Compare o primeiro bit:

A₃ = 1
B₃ = 0

Como no bit mais significativo **A** já é maior:

A > B

Não precisa olhar os outros bits.

9.3 Exemplo em que precisa continuar

A = 1010
B = 1001

Compare:

A3 = 1, B3 = 1 -> empatou
A2 = 0, B2 = 0 -> empatou
A1 = 1, B1 = 0 -> A é maior

Logo:

A > B

9.4 Igualdade com XNOR

Dois bits são iguais quando:

0 e 0
ou
1 e 1

Isso é XNOR.

Para números de 4 bits:

A = B quando todos os bits são iguais

Então:

$EQ = (A3 \text{ XNOR } B3) \cdot (A2 \text{ XNOR } B2) \cdot (A1 \text{ XNOR } B1) \cdot (A0 \text{ XNOR } B0)$

Em palavras:

compara bit a bit
se todos forem iguais, A = B

9.5 Comparador de 2 bits

Para A = A1A0 e B = B1B0:

$EQ = (A1 \text{ XNOR } B1) \cdot (A0 \text{ XNOR } B0)$

Para A > B:

$A > B$ se $A_1 > B_1$
ou se $A_1 = B_1$ e $A_0 > B_0$

Em expressão:

$GT = A_1 \cdot B_1' + (A_1 \text{ XNOR } B_1) \cdot A_0 \cdot B_0'$

Para $A < B$:

$LT = A_1' \cdot B_1 + (A_1 \text{ XNOR } B_1) \cdot A_0' \cdot B_0$

Você não precisa decorar as expressões grandes.

O que precisa memorizar é o procedimento:

compare do bit mais significativo para o menos significativo
o primeiro bit diferente decide
se nenhum bit for diferente, os números são iguais

9.6 Cuidado com comparação signed e unsigned

O mesmo padrão de bits pode ter interpretações diferentes.

Exemplo em 4 bits:

$A = 1010$
 $B = 0111$

Sem sinal:

$A = 10$
 $B = 7$
 $A > B$

Com sinal em complemento de 2:

$A = -6$
 $B = +7$
 $A < B$

Então, em prova, sempre veja se o enunciado fala:

sem sinal
com sinal
complemento de 2

Se não falar, muitas questões de comparador básico assumem sem sinal.

10. Comparação Usando Subtração

Uma forma comum de comparar é fazer:

$A - B$

Depois analisar o resultado.

Para comparação sem sinal:

se $A - B = 0 \rightarrow A = B$
se não houve borrow $\rightarrow A \geq B$
se houve borrow $\rightarrow A < B$

Lembre da regra da subtração por complemento de 2:

carry-out final 1 $\rightarrow$ não houve borrow
carry-out final 0 $\rightarrow$ houve borrow

Exemplo:

$A = 0110$
 $B = 0011$

$A - B$:

$0110 - 0011 = 0011$
carry-out final = 1

Logo:

$A > B$

Outro exemplo:

$A = 0011$
 $B = 0101$

$A - B$:

$0011 - 0101 = 1110$
carry-out final = 0

Logo, sem sinal:

$A < B$

11. Shifter

11.1 O que é

Um shifter é um circuito que desloca bits.

Ele pode deslocar:

para a esquerda
para a direita

E pode preencher a vaga que sobra com:

0
bit de sinal
bit que saiu do outro lado

Dependendo do preenchimento, temos tipos diferentes de deslocamento.

11.2 Shift lógico para a esquerda

No shift lógico para a esquerda, os bits andam para a esquerda e entra 0 à direita.

Exemplo em 4 bits:

$1011 \ll 1 = 0110$

Rastreio:

antes: 1 0 1 1
depois: 0 1 1 0

O bit mais à esquerda saiu e foi descartado.

Para números sem sinal, deslocar uma posição à esquerda costuma equivaler a multiplicar por 2, se não houver estouro.

Exemplo:

```
0011 = 3  
0011 << 1 = 0110 = 6
```

Mas:

```
1001 = 9  
1001 << 1 = 0010
```

Em 4 bits, o bit que saiu foi perdido.

Então não dá para dizer simplesmente que sempre multiplica por 2 sem checar a perda de bits.

11.3 Shift lógico para a direita

No shift lógico para a direita, os bits andam para a direita e entra 0 à esquerda.

Exemplo:

```
1011 >> 1 = 0101
```

Rastreio:

```
antes:  1 0 1 1  
depois: 0 1 0 1
```

Para números sem sinal, deslocar uma posição à direita costuma equivaler a dividir por 2, descartando o resto.

Exemplo:

```
1010 = 10  
1010 >> 1 = 0101 = 5
```

Outro:

```
1011 = 11  
1011 >> 1 = 0101 = 5
```

11 / 2 = 5 com resto 1.

O bit descartado representa esse resto que foi perdido.

11.4 Shift aritmético para a direita

O shift aritmético para a direita é usado com números com sinal em complemento de 2.

Ele preserva o bit de sinal.

Exemplo:

1011

Em 4 bits com sinal:

1011 = -5

Shift lógico para a direita:

1011 -> 0101

Isso vira +5, o que muda o sinal.

Shift aritmético para a direita:

1011 -> 1101

O bit de sinal era 1, então entra 1 à esquerda.

Em complemento de 2:

1101 = -3

Isso se comporta como uma divisão aproximada por 2 mantendo o sinal.

11.5 Rotate

Em rotate, o bit que sai de um lado entra pelo outro.

Exemplo de rotação para a esquerda:

1011 -> 0111

O 1 que saiu da esquerda entrou na direita.

Rotate é importante em algumas arquiteturas, mas para esta prova, o mais essencial é:

shift lógico
shift aritmético
diferença entre preencher com 0 e preservar sinal

11.6 Como um shifter aparece em circuito

Para deslocamento fixo de 1 bit, pode ser só ligação de fios:

```
entrada:  A3 A2 A1 A0
SHL 1:    A2 A1 A0 0
LSR 1:    0  A3 A2 A1
ASR 1:    A3 A3 A2 A1
```

Para escolher entre vários deslocamentos, o circuito usa multiplexadores.

Um shifter que consegue deslocar várias posições rapidamente é chamado de:

barrel shifter

Ideia:

controle escolhe quantas posições deslocar
multiplexadores selecionam quais bits chegam na saída

12. ULA

12.1 O que é

ULA significa:

Unidade Lógica e Aritmética

Em inglês:

ALU - Arithmetic Logic Unit

A ULA é o bloco que executa operações como:

soma
subtração
AND
OR
XOR
comparação
deslocamento

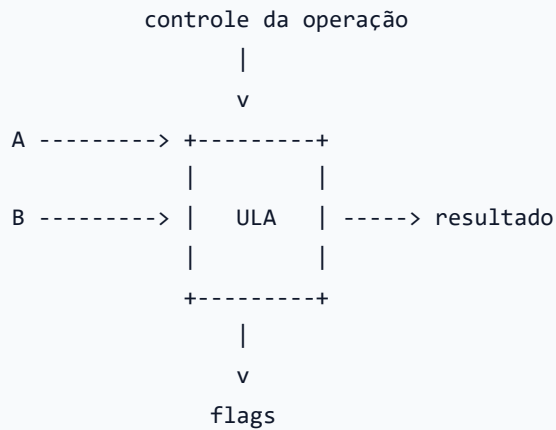
Ela recebe:

operandos A e B
sinais de controle

E gera:

resultado
flags

12.2 Diagrama mental



Flags comuns:

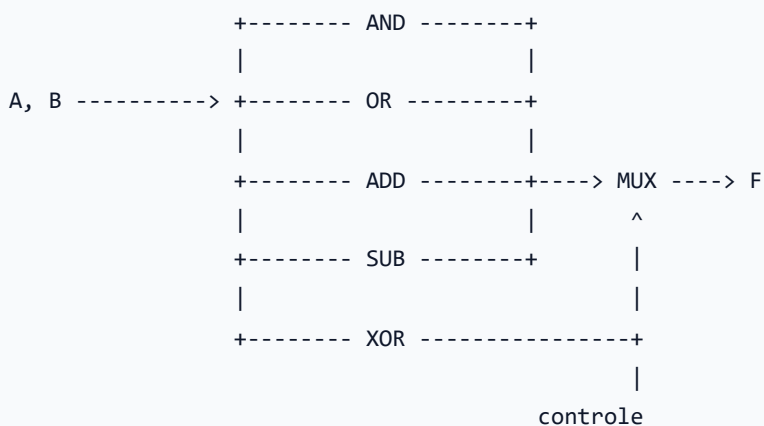
Z -> zero
N -> negativo
C -> carry
V -> overflow

12.3 ULA como seleção entre blocos

Uma forma simples de enxergar a ULA é:

calcula várias operações em paralelo
um MUX escolhe qual resultado sai

Exemplo:



O controle escolhe a operação.

Exemplo de tabela:

Controle	Operação	Saída F
000	AND	$A \text{ AND } B$
001	OR	$A \text{ OR } B$
010	ADD	$A + B$
011	SUB	$A - B$
100	XOR	$A \text{ XOR } B$
101	SLT/comparação	1 se $A < B$
110	Shift left	$A \ll 1$
111	Shift right	$A \gg 1$

Essa tabela é só um exemplo.

Cada arquitetura pode escolher códigos de controle diferentes.

O importante é:

os sinais de controle dizem qual operação a ULA deve entregar

12.4 Soma e subtração na mesma ULA

A ULA costuma reaproveitar o mesmo somador para soma e subtração.

Use o sinal:

SUB

Quando $SUB=0$:

$B_{\text{entrada}} = B$
 $C0 = 0$
 $F = A + B$

Quando $SUB=1$:

$B_{\text{entrada}} = B'$
 $C0 = 1$
 $F = A + B' + 1$
 $F = A - B$

Ou seja:

o mesmo somador faz as duas operações

12.5 Flags da ULA

Zero

A flag zero vale 1 quando o resultado é zero.

$Z = 1$ se $F = 0000$
 $Z = 0$ caso contrário

Pode ser implementada com NOR dos bits do resultado:

$Z = (F_3 + F_2 + F_1 + F_0)'$

Negative

A flag negative normalmente copia o bit mais significativo do resultado.

Em 4 bits:

$N = F_3$

Em complemento de 2:

$F_3 = 1 \rightarrow$ resultado negativo
 $F_3 = 0 \rightarrow$ resultado não negativo

Carry

Em soma sem sinal:

$C =$ carry-out final

Em subtração sem sinal implementada como $A + B' + 1$:

carry-out final 1 $\rightarrow$ não houve borrow
carry-out final 0 $\rightarrow$ houve borrow

Cuidado: algumas arquiteturas definem flag de carry/borrow de formas específicas. Para prova de circuitos digitais, entenda a regra conceitual.

Overflow

Overflow é usado para números com sinal em complemento de 2.

Na soma:

dois positivos deram negativo -> overflow
dois negativos deram positivo -> overflow

Na subtração:

A - B tem overflow quando A e B têm sinais diferentes
e o sinal do resultado é diferente do sinal de A

Também pode ser calculado por:

V = carry que entra no bit de sinal XOR carry que sai do bit de sinal

13. Exemplos Resolvidos

13.1 Subtração com somador

Calcule:

0101 - 0010

Em decimal:

5 - 2 = 3

Pelo circuito:

B = 0010
B' = 1101
B' + 1 = 1110

Some:

0101
+ 1110

1 0011

Resultado em 4 bits:

0011

Carry-out final:

1

Sem sinal:

não houve borrow

13.2 Comparador sem sinal

Compare:

A = 1001

B = 1010

Compare do bit mais significativo:

A3 = 1, B3 = 1 -> empatou

A2 = 0, B2 = 0 -> empatou

A1 = 0, B1 = 1 -> B é maior

Logo:

A < B

13.3 Shift lógico

Faça shift lógico para a esquerda:

0011 << 1

Resultado:

0110

Sem sinal:

3 virou 6

Agora:

1100 << 1 = 1000

O bit da esquerda foi perdido.

Então, em largura fixa, shift pode causar perda de informação.

13.4 Shift aritmético para a direita

Faça shift aritmético para a direita:

1010

O bit de sinal é:

1

Então entra 1 à esquerda:

1010 -> 1101

Em complemento de 2:

1010 = -6

1101 = -3

13.5 ULA com controle

Considere uma ULA de 4 bits com a tabela:

Controle	Operação
00	AND
01	OR
10	ADD
11	SUB

Entradas:

A = 0101

B = 0011

controle = 10

Controle 10 significa:

ADD

Então:

```
F = A + B  
F = 0101 + 0011  
F = 1000
```

Se o controle fosse 11:

```
SUB
```

Então:

```
F = A - B  
F = 0101 - 0011  
F = 0010
```

14. Como Questões Costumam Cobrar

14.1 Subtrator

Pergunta típica:

Como implementar $A - B$ usando um somador?

Resposta:

```
inverter B  
colocar carry-in inicial igual a 1  
somar  $A + B' + 1$ 
```

14.2 Somador/subtrator

Pergunta típica:

Qual é a função do sinal SUB em um somador/subtrator?

Resposta:

```
SUB=0 -> passa B normal e C0=0  
SUB=1 -> inverte B e C0=1
```

14.3 Comparador

Pergunta típica:

Como decidir se $A > B$?

Resposta:

compare do bit mais significativo para o menos significativo
o primeiro bit diferente decide

14.4 Shifter

Pergunta típica:

Qual é a diferença entre shift lógico para a direita e shift aritmético para a direita?

Resposta:

lógico -> entra 0 à esquerda
aritmético -> replica o bit de sinal

14.5 ULA

Pergunta típica:

O que os sinais de controle da ULA fazem?

Resposta:

selecionam qual operação será enviada para a saída

15. Erros Comuns

15.1 Achar que subtrator precisa sempre ser um circuito separado

Na prática, a subtração costuma reaproveitar o somador:

$$A - B = A + B' + 1$$

15.2 Confundir carry-out em subtração sem sinal

Em subtração por complemento de 2:

```
carry-out 1 -> não houve borrow  
carry-out 0 -> houve borrow
```

Isso é contraintuitivo no começo.

15.3 Comparar signed como se fosse unsigned

Exemplo:

```
1010 > 0111 sem sinal  
1010 < 0111 com sinal em complemento de 2
```

O enunciado manda.

15.4 Fazer shift aritmético preenchendo com zero

Para número com sinal, shift aritmético para a direita replica o bit de sinal.

Exemplo:

```
1011 ASR 1 = 1101
```

Não é:

```
0101
```

15.5 Achar que ULA é sequencial

Uma ULA básica é combinacional.

Ela não guarda estado sozinha.

Quem guarda valores são registradores e flip-flops.

16. Exercícios

16.1 Conceituais

1. O que é borrow em uma subtração binária?
2. Qual é a diferença entre half-subtractor e full-subtractor?
3. Como implementar $A - B$ usando um somador?
4. Em um somador/subtrator, o que acontece quando $SUB=1$?
5. Qual é a regra prática para comparar dois números binários sem sinal?

6. Qual é a diferença entre shift lógico para a direita e shift aritmético para a direita?
7. O que é uma ULA?
8. Cite quatro operações que uma ULA pode realizar.

16.2 Aplicação direta

9. Em um half-subtractor, calcule D e $Bout$ para $A=0$, $B=1$.
10. Em um full-subtractor, calcule D para $A=1$, $B=1$, $Bin=1$.
11. Faça $0110 - 0010$ em 4 bits usando complemento de 2.
12. Faça $0010 - 0101$ em 4 bits usando complemento de 2 e diga se houve borrow sem sinal.
13. Compare sem sinal: $A=1011$, $B=1001$. Qual é maior?
14. Compare sem sinal: $A=0110$, $B=0110$. Quais saídas do comparador devem ficar ativas?
15. Faça shift lógico para a esquerda de 0101 .
16. Faça shift lógico para a direita de 1010 .
17. Faça shift aritmético para a direita de 1010 .
18. Uma ULA tem controle $00=AND$, $01=OR$, $10=ADD$, $11=SUB$. Para $A=0110$, $B=0011$, controle 11 , qual é a saída?

16.3 Questões mais parecidas com prova

19. Em um somador/subtrator de 4 bits, a entrada de cada bit de B passa por uma XOR com o sinal SUB , e o carry-in inicial também recebe SUB . Qual operação ocorre quando $SUB=0$ e quando $SUB=1$?
20. Em uma subtração sem sinal implementada como $A + B' + 1$, o carry-out final foi 0 . O que isso indica?
21. Em complemento de 2 com 4 bits, calcule $0111 - 1111$. Houve overflow com sinal?
22. Em uma ULA, a flag Z vale 1 quando o resultado é 0000 . Qual porta lógica poderia detectar isso a partir de todos os bits do resultado?
23. Um shifter recebe 1101 . Qual é o resultado de shift lógico para a direita e shift aritmético para a direita?
24. Um comparador sem sinal recebe $A=0100$ e $B=1000$. Qual saída fica ativa: $A>B$, $A=B$ ou $A<B$?

17. Gabarito

1. Borrow é o empréstimo necessário quando a coluna atual não consegue subtrair B de A .
2. Half-subtractor não tem borrow de entrada. Full-subtractor tem Bin e calcula $A - B - Bin$.
3. Inverta B , some 1 e use o somador: $A - B = A + B' + 1$.
4. B é invertido e o carry-in inicial vira 1. O circuito calcula $A - B$.

5. Compare do bit mais significativo para o menos significativo. O primeiro bit diferente decide.
6. Shift lógico para a direita coloca 0 à esquerda. Shift aritmético para a direita replica o bit de sinal.
7. ULA é a Unidade Lógica e Aritmética, bloco combinacional que executa operações sobre operandos.
8. Exemplos: soma, subtração, AND, OR, XOR, comparação, shift left, shift right.
9. $0 - 1$ gera $D=1$ e $Bout=1$.
10. $1 - 1 - 1 = -1$, então $D=1$.
11. $B=0010$, $B'=1101$, $B'+1=1110$. $0110 + 1110 = 1\ 0100$. Resultado em 4 bits: 0100 .
12. $B=0101$, $B'=1010$, $B'+1=1011$. $0010 + 1011 = 1101$, carry-out 0. Houve borrow sem sinal.
13. $A=1011$ é maior, pois os bits mais significativos empatam até $A1=1$ e $B1=0$.
14. Apenas $A=B$ fica ativa.
15. $0101 \ll 1 = 1010$.
16. Shift lógico para a direita: $1010 \rightarrow 0101$.
17. Shift aritmético para a direita: $1010 \rightarrow 1101$.
18. Controle 11 é subtração. $0110 - 0011 = 0011$.
19. $SUB=0$ faz $A+B$. $SUB=1$ faz $A+B'+1$, ou seja, $A-B$.
20. Indica que houve borrow, isto é, $A < B$ na interpretação sem sinal.
21. $1111=-1$, então $0111 - 1111 = 7 - (-1) = 8$, que não cabe em 4 bits com sinal. O resultado fica 1000 , logo houve overflow.
22. Uma NOR de todos os bits do resultado: $Z = (F3 + F2 + F1 + F0)'$.
23. Shift lógico para a direita: 0110 . Shift aritmético para a direita: 1110 .
24. $A=4$ e $B=8$, então $A < B$ fica ativa.

18. O Que Memorizar

Half-subtractor:

$$D = A \text{ XOR } B$$

$$Bout = A' \cdot B$$

Full-subtractor:

$$D = A \text{ XOR } B \text{ XOR } Bin$$

Subtração com somador:

$$A - B = A + B' + 1$$

Somador/subtrator:
B_entrada = B XOR SUB
C0 = SUB

SUB=0 -> soma
SUB=1 -> subtração

Subtração sem sinal:
carry-out 1 -> sem borrow
carry-out 0 -> com borrow

Comparador sem sinal:
compare do bit mais significativo para o menos significativo

Shift lógico à direita:
entra 0 à esquerda

Shift aritmético à direita:
replica o bit de sinal

ULA:
bloco combinacional que escolhe operação por sinais de controle

19. Plano De Estudo Para Esta Aula

Sugestão:

1. Ler seções 2 a 8 para entender subtração.
2. Refazer os exemplos 6, 7 e 8 no papel.
3. Ler comparador e fazer 5 comparações unsigned.
4. Treinar shift lógico e aritmético com 6 padrões de 4 bits.
5. Ler ULA pensando nela como um MUX entre operações.
6. Resolver exercícios 9 a 24.

Se o tempo estiver curto, priorize:

11, 12, 18, 19, 20, 21, 23 e 24

Esses pegam os pontos com maior chance de confusão:

- subtração com complemento de 2
- borrow
- overflow
- controle da ULA
- shift lógico vs aritmético

20. Conexão Com A Próxima Aula

Na próxima aula, vamos continuar em aritmética digital, mas agora com representação de números não inteiros ou números com escala.

O tema será:

- ponto fixo
- ponto flutuante

Isso conecta com:

- como representar frações em binário
- como interpretar escala
- por que ponto flutuante usa sinal, expoente e mantissa
- quais erros de precisão podem aparecer

Depois desta aula, a ideia é que você já enxergue melhor a base da ULA:

- somar
- subtrair
- comparar
- deslocar
- selecionar operação